Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Nate Cheek	aspenchk@udel.edu
Friend (optional)	Justin	justincb@udel.edu

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea

and a paragraph describing how that paper supports your design concept.
        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider the project below.
        - Create an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the chempy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.

I will either do the Electric and computer engineering department project, or something more like Cryptographic hashes like I did a little last year in my Cybersecurity class. I'd probably do RSA, and like one of the old SHA ones, maybe a third if I still need pages.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

There is a good example site at: https://coddy.tech/tools/hash-generator

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

People could use the site to generate hashes for fun.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

To generate a message hash with different cases based on options.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Message and the hash type, maybe a number for the random part.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

It will take the message input and calculate the hash based of it in the format selected.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The hash and the decryption key.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Message, Hash Function, selected key.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Need: message *Ignore: Spelling errors in the message, newline and tab characters*

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

Coding the hash functions correctly, getting the pages to display correctly with the correct information.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will be using an LLM. I have already done things like this in a course last year, so I will be comparing/referencing the code I still have from doing that. For the hashing programs I haven't done before I will be referencing sources to see how they work.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I want to understand how hashes work better, and get experience on how to make a working website.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to make a simple webtool to create several different encryption / decryption programs (like RSA or MD-5). What things should I include, while keeping it simple?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

It suggests most of the things I already said earlier in the document, but also says including a copy to clipboard button.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

the website is supposed to be relatively simple, with 5 pages for different things

*it suggested lowering the amount of algorithms included, which is what I was going to do based off its first thing.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

Basically none, I just hadn't written down the short conversation I had with a classmate so I had to ask the AI. I guess I will probably include a copy button, but I was already planning to do 90% of the stuff it said, and the other 10% is not within my wanted project scope.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Drafter documentation, my past code, website that explains the crypto algorithms, office hours, etc…

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will probably trace through the code alongside drafter documentation if I can't get it to work. If I still cant figure it out I will go to the discord or office hours for help

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

I want to make a simple webtool to create several different encryption / decryption programs (like RSA or MD-5). What things should I include, while keeping it simple?

ChatGPT said:
I would recommend making it a simple encryption and cryptography toolkit: a website where users can select an algorithm, enter some text, choose whether to encrypt or decrypt (where applicable), and get an output.

The key is to avoid making it too complicated. Focus on a clean interface and a handful of useful algorithms.

1. Core features
I'd organize the website around these features:

Feature

Description

Priority
Algorithm selector
Choose RSA, AES, MD5, SHA-256, etc.
Essential
Input box
Enter text or paste encrypted data
Essential
Encrypt / Decrypt
Select the operation
Essential
Output box
Display the result
Essential
Copy button
Copy the result to clipboard
Essential
Clear button
Clear input and output
Essential
Algorithm information
Briefly explain the selected algorithm
Useful
Error handling

Explain invalid input or incorrect keys

Essential

Download result

Save output as a text file

Optional

2. Algorithms to include
I'd start with around 6 algorithms, split into three categories.

Encryption and decryption
These use keys and can reverse the encryption when the correct key is provided.

Secure Voice and Data Systems | Tait Communications Blog

AES

Symmetric encryption. Uses the same secret key to encrypt and decrypt.

Recommended
RSA Encryption : Beginners guide. What the heck is Encryption anyway? | by Vishwas Adhikari | Medium

RSA

Asymmetric encryption. Uses a public key and a private key.

Caesar cipher | PPT

Caesar Cipher

A simple substitution cipher that shifts letters by a specified number.

Vigenere

Vigenère Cipher

A classical cipher that uses a keyword to encrypt and decrypt text.

Hashing (one-way)
These algorithms generate a hash from input text. They cannot be decrypted.

MD5 – Produces a 128-bit hash. Useful for demonstrating older hashing techniques, but not

secure for modern password storage.

SHA-256 – Produces a 256-bit hash and is widely used in modern cryptographic applications.

I'd include both because they make a good comparison.

Encoding (not encryption)
These are useful additions for a general-purpose cryptography toolkit:

Base64

Hexadecimal

Binary

These aren't encryption algorithms, but they're useful for converting data between formats.

3. Suggested website layout
Keep everything on one main page rather than creating separate pages for every algorithm.

CryptoTool
Demo layout
A simple encryption and hashing toolkit

Select an algorithm

Operation

Encrypt
Decrypt
Hash
Input text

Hello World!
Key / Password
Enter encryption key (if required)

Output

Your result will appear here...

This is a UI mockup; it does not perform actual encryption.

4. Small extras that make it better
You don't need all of these, but I'd consider:

Algorithm descriptions: A small information icon explaining how each algorithm works.

Example input: A button that fills the input with sample text.

Key generator: Automatically generate a random encryption key.

Show/hide key: Allow users to reveal or hide their key.

Character counter: Show the number of characters in the input.

History: Keep a small list of previous operations during the current session.

5. Technology recommendations
For a relatively simple web project:

Technology

Purpose

HTML

Website structure

CSS

Styling and layout

JavaScript

Encryption functionality

Web Crypto API

AES, RSA, SHA-256 and secure random generation

You can implement classical ciphers yourself in JavaScript for educational purposes.

One important distinction: MD5 and SHA-256 are hashing algorithms, not encryption algorithms. They are one-way operations. Also, MD5 should only be included for educational or compatibility purposes, not security.

6. My recommended final scope
If I were building this as a small project, I'd use this feature list:

CryptoTool – MVP

Recommended scope
Algorithms

AES

RSA

Caesar Cipher

Vigenère Cipher

MD5

SHA-256

Base64

Interface

Single-page design

Dropdown algorithm selector

Encrypt / Decrypt / Hash controls

Input and output text areas

Key input where applicable

Copy and clear buttons

Additional functionality

Short algorithm explanations

Example input button

Random key generator

Basic error handling

My biggest recommendation: Keep the website focused on the actual cryptographic functionality rather than adding too many features. A user should be able to open the website, select an algorithm, enter text, click a button, and get a result within seconds.

That would make a clean, useful, and achievable project without overengineering it.

Log in for personalized, step-by-step guidance.

Log in

Sign up for free
You said:
the website is supposed to be relatively simple, with 5 pages for different things

ChatGPT said:
In that case, I'd structure it as a 5-page cryptography website, with each page having a distinct purpose. Keep the navigation consistent across all five pages and avoid cramming every tool into one interface.

Here's the structure I'd recommend:

Suggested 5-page website structure
1. Home

Essential
Evervault - Awwwards Honorable Mention

Brief introduction to the website.

Explain what encryption and decryption are.

Display links to the other four pages.

Feature a few popular algorithms.

2. Encryption

CryptoVault – Enterprise-Grade Encryption/Decryption Tool

Select an encryption algorithm.

Enter original text.

Enter or generate an encryption key.

Click Encrypt.

Display encrypted output.

Include AES, RSA, Caesar Cipher and Vigenère Cipher.

3. Decryption

Text Encryptor / Decryptor — UtilityCove | Free Online Tool

Select the encryption algorithm.

Paste encrypted text.

Enter the necessary key.

Click Decrypt.

Display the original text.

4. Hashing

The Utility Belt | 25+ Standalone Developer Tools

Enter text.

Select a hashing algorithm.

Generate hash.

Copy the hash.

Include MD5, SHA-1 and SHA-256.

Briefly explain that hashes cannot be decrypted.

5. About / Learn

RSA Encryption : Beginners guide. What the heck is Encryption anyway? | by Vishwas Adhikari | Medium

Explain what cryptography is.

Explain encryption versus hashing.

Give a short explanation of each algorithm.

Include a few real-world uses of cryptography.

Navigation bar

I'd keep the navigation bar simple:

CryptoTool

Home

Encrypt

Decrypt

Hashing

About

A few design recommendations
Use the same color scheme and typography on every page.

Keep the encryption and decryption interfaces nearly identical.

Avoid excessive animations and complicated transitions.

Use cards to introduce algorithms.

Make the website responsive for mobile and desktop.

One important consideration
I'd keep RSA and AES as the main modern encryption demonstrations, and Caesar and Vigenère as simpler educational examples. For hashing, MD5 and SHA-256 are good demonstrations, but MD5 shouldn't be used for real security.

One question: Is this website primarily a school/educational project, or are you aiming to make a practical cryptography tool that people could actually use? That would influence how much functionality each page needs.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.